



Comparing Cloud-Native MLOps Platforms: AWS, Azure, GCP, and Databricks

Sergio Moreschini^{ID}, David Hästbacka^{ID}, and Davide Taibi

From the Editor

Machine learning operations (MLOps) has rapidly evolved from a marginal concept to a pivotal consideration for any enterprise implementing ML at scale. Cloud providers have rushed to fill this gap, each offering its own take on what MLOps should be.

This article compares the MLOps offerings of Amazon Web Services (AWS), Azure, Google Cloud Platform (GCP), and Databricks. Their analysis reveals the diverse perspectives guiding each vendor's approach, assisting practitioners in identifying suitable tradeoffs for their specific contexts. — *Davide Taibi*

OPERATIONALIZING MACHINE LEARNING (ML) in production is challenging. Unlike traditional software, ML-based systems depend not only on code but also on data, models, and evolving pipelines. Ensuring reproducibility, scalability, and governance across these elements requires practices beyond classical software engineering. To address these challenges, the community has converged on ML operations (MLOps), an extension of DevOps practices to ML, becoming the reference paradigm for building and operating ML systems at scale.

However, MLOps is not a monolithic concept. Different entities promote different perspectives on what MLOps is and what it should achieve. These insights reflect a range of priorities, including cultural continuity with DevOps, automation maturity, and structural separation of responsibilities. Perspectives like these lead to differences in how vendors, such as Amazon Web Services (AWS)^a and Microsoft Azure,^b design their platforms.

- One of these perspectives¹ highlights MLOps as an evolution of DevOps, extending the infinity

loop with ML-specific steps such as continuous training (CT) and model validation. This stresses cultural and procedural continuity: MLOps as DevOps adapted to new workflows and artifacts.

- A second perspective, exemplified by Google Cloud Platform (GCP),^c frames MLOps around levels of automation, guiding organizations from manual operations to fully automated pipelines.²
- A third perspective, advanced by Databricks,^d introduces a division into DataOps, ModelOps, and DevOps, highlighting the need

Digital Object Identifier 10.1109/MS.2025.3646505
Date of current version: 26 February 2026

^aAWS: <https://aws.amazon.com>.

^bAzure: <https://azure.microsoft.com/>.

^cGoogle Cloud: <https://cloud.google.com>.

^dDatabricks: <https://www.databricks.com>.

to orchestrate data governance, model lifecycle management, and continuous delivery as distinct but interdependent responsibilities.

These perspectives are not purely theoretical; they underpin how cloud platforms have positioned themselves as MLOps enablers. AWS, Azure, GCP, and Databricks all provide managed services that claim comprehensive coverage of the MLOps lifecycle, but their priorities and integration models diverge. For practitioners, this results in a fragmented landscape where platform selection involves difficult tradeoffs related to openness, governance, scalability, cost, and the risk of vendor lock-in.

We provide a comparative analysis of AWS, Azure, GCP, and Databricks as MLOps platforms, using a common set of evaluation characteristics to make their strengths, limitations, and underlying assumptions explicit. Rather than identifying a single “best” solution, our goal is to help practitioners understand the tradeoffs each platform entails and select the one that best aligns with their context. As we show later, platform selection ultimately hinges on five recurring dimensions—performance, cost, openness, data management, and learning curve—which structure our discussion of tradeoffs.

Foundations of MLOps

Although cloud vendors promote different interpretations of MLOps, a shared set of principles underpins modern ML operations. These principles extend DevOps practices to reflect the lifecycle of data, models, and pipelines.

- **Versioning:** ML systems require version control for code, datasets, preprocessing steps,

and model artifacts. Consistent versioning ensures traceability across experiments and enables reliable reproduction or rollback when performance degrades.

- **Automation:** Beyond continuous integration/continuous delivery (CI/CD), MLOps introduces *CT* and *continuous monitoring (CM)*, allowing pipelines to respond automatically to data drift, model degradation, and operational signals. These automated loops transform ML workflows from one-off deployments into adaptive systems.
- **Governance:** Production ML requires security, access control, auditability, lineage, and policy enforcement, particularly in regulated domains. Governance mechanisms ensure that data and models evolve safely and transparently across teams and environments.
- **Scalability:** ML workloads fluctuate with data size, training frequency, and model complexity. Effective MLOps implementations rely on elastic compute resources, distributed training, and scalable inference endpoints to support changing demands.

When implemented in the cloud, additional factors emerge. *Cost* is influenced by automation overhead, pipeline orchestration, and idle resource management, while *openness* determines how easily organizations integrate external tools, avoid vendor lock-in, and support hybrid or multi-cloud deployments. These foundations clarify the operational capabilities that cloud platforms must support.

Comparison of Platforms

The foundations introduced in the previous section provide the conceptual

basis for our comparison. However, to enable a clearer cross-vendor assessment, we translate these principles into a set of concrete capability-based criteria. Table 1 provides a summary of these operational criteria, including native support for model and dataset versioning, pipeline orchestration, CT & CM, responsible AI functionalities, distributed training, autoscaling inference, enterprise-grade compliance, openness toward external tools, and cross-cloud portability. Collectively, these dimensions offer a comprehensive view of the capabilities supported by each vendor, highlighting differences in maturity, integration depth, and overall MLOps philosophy.

AWS SageMaker

The AWS approach to MLOps can be summarized as *S3-first*. Amazon S3 was one of the earliest services launched by AWS, and it quickly became the cornerstone of the cloud. Over time, most other AWS services have been designed to integrate around it, turning S3 into the de facto hub for data and artifacts. In the MLOps context, this means that pipelines naturally begin with data stored in S3, and the rest of the workflow is layered on top, using services such as SageMaker, CodePipeline, Lambda, and EMR. Rather than positioning MLOps as a standalone offering, AWS extends its long-standing storage-first philosophy to ML.

In terms of version control, SageMaker provides a Model Registry for tracking models, while data versioning typically requires integration with DVC.⁴ Another significant integration within AWS is with MLflow, a tool that allows for experiment tracking and artifact management. This provides practitioners with an open source option within the managed AWS ecosystem.

Table 1. Common comparison criteria across cloud MLOps platforms.

Criterion	AWS	Azure	GCP	Databricks
Native model registry	✓	✓	✓	✓
Native dataset versioning	X (via DVC)	✓	✓	✓(Delta Lake)
Pipeline orchestration	✓(SageMaker Pipelines)*	✓(Azure ML Pipelines + DevOps)	✓(Vertex Pipelines)	✓(Lakehouse Jobs)
CT	✓	✓	✓	✓
Responsible/explainable AI tools	✓(Clarify)	✓(Azure Policy ³)	✓(RAI Toolkit)	✓(Unity Catalog + governance)
Distributed training	✓	✓	✓	✓
Serverless/autoscaling inference	✓	✓	✓	✓
Cross-cloud portability	X	X	X	✓
Enterprise compliance strength	✓	✓✓✓	✓✓	✓✓
OSS integration (MLflow, frameworks)	✓	✓	✓	✓✓✓

*SageMaker Pipelines: <https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines.html>.

X: not available or not natively available; ✓: available/positive; ✓✓: more than positive; ✓✓✓: best.

Automation is orchestrated primarily through SageMaker Pipelines,^e which manage data preprocessing, training, evaluation, and deployment. These pipelines integrate with CodePipeline and CodeBuild,^f extending DevOps CI/CD into the ML lifecycle. With Model Monitor,^g AWS closes the loop by enabling CT and monitoring, retriggering pipelines when drift or degradation is detected. This establishes AWS automation as a system of feedback loops, from S3-based data changes through retraining and redeployment.

AWS provides robust governance features. SageMaker Clarify supports bias detection and model explainability,⁵ while Identity and Access

Management (IAM) and CloudTrail ensure secure access and auditing of application programming interface (API) calls. Together with the Model Registry, these services enable traceability and compliance across regulated domains.

SageMaker supports distributed training on GPUs and AWS's own Trainium and Inferentia accelerators, designed to lower the cost of large-scale ML. For inference, serverless endpoints automatically scale with workload demand, allowing teams to grow seamlessly from prototype to production.

AWS follows a pay-as-you-go model that provides fine-grained control but can become complex when multiple services are combined. Hidden costs from data transfer, orchestration, or idle resources are well-documented concerns in multiservice pipelines.

While SageMaker supports popular open source frameworks (for example, TensorFlow and PyTorch),

the underlying workflows—storage, identity, monitoring, and orchestration—are tightly integrated with AWS infrastructure. This integration results in a form of switching cost or vendor lock-in because porting to a different cloud often requires rearchitecting many pipeline components.

Azure ML

Azure embodies an *enterprise DevOps* mentality. Its ML ecosystem is designed to extend the practices of Azure DevOps into the ML lifecycle, making it particularly attractive to organizations already invested in Microsoft environments. The centerpiece is Azure ML, which combines notebook-based development with Azure ML Studio, a user-friendly platform that supports both code-first and no-code workflows.

For version control, Azure ML integrates directly with Git repositories for code and pipeline tracking while

^eSageMaker Pipelines: <https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines.html>.

^fAWS CodeBuild: <https://docs.aws.amazon.com/codebuild/latest/userguide/welcome.html>.

^gSageMaker Model Monitor: <https://docs.aws.amazon.com/sagemaker/latest/dg/model-monitor-mlops.html>.

supporting dataset versioning and a Model Registry to manage model life-cycle and approvals. This integration extends naturally into Azure DevOps Repos and GitHub, ensuring that ML artifacts are managed with the same rigor as software components.

Automation is orchestrated through Azure ML Pipelines, which cover data preprocessing, training, deployment, and monitoring. These pipelines are deeply integrated with Azure DevOps, enabling seamless CI/CD extensions into CT and monitoring. Retraining can be triggered automatically based on monitoring events, and pipelines can be designed either in code or visually within ML Studio. This alignment ensures that feedback loops are embedded within enterprise DevOps workflows.⁶

Governance is Azure's strongest differentiator through Azure Policy.³ The platform includes a broad range of compliance certifications (ISO, SOC, HIPAA, and GDPR) and dedicated responsible AI dashboards for fairness, interpretability, and transparency.

For scalability, Azure ML leverages Azure Kubernetes Service (AKS) for deployment, with support for distributed GPU/CPU clusters during training. Moreover, integration with other Azure cloud services makes scaling ML workloads within enterprise ecosystems straightforward.

Cost management is facilitated by enterprise-oriented pricing models, calculators, and budgeting tools. However, efficiency is optimized when organizations are already committed to Microsoft's broader services; using Azure ML outside of this ecosystem may introduce overheads.

When it comes to openness, Azure supports popular frameworks such as TensorFlow, PyTorch, and scikit-learn, but workflows are optimized for Microsoft services. This

reduces cross-cloud portability, even though individual models and artifacts can be exported.

Google Cloud—Vertex AI

GCP advances an *automation maturity* mindset, framing MLOps as a progression from manual workflows to fully automated feedback-driven pipelines. Vertex AI consolidates Google's ML offerings into one managed environment. The Vertex AI Workbench acts as the central workspace for both notebook-driven development and AutoML, supporting seamless transitions between custom code and automated pipelines.⁷

For version control, Vertex AI natively supports dataset and model versioning, with metadata tracking integrated into pipelines. The Experiments API allows teams to log parameters, metrics, and artifacts, ensuring traceability across runs.

Automation is GCP's defining strength. Vertex AI Pipelines orchestrate end-to-end workflows with support for TensorFlow, PyTorch, and scikit-learn for custom training. On the other hand, it is also possible to build an MLOps system with Vertex AI AutoML and Pipeline where AutoML handles feature engineering, model selection, and hyperparameter tuning, significantly reducing the need for manual intervention. Integration with Cloud Build^h and Cloud Run Functions^{i,4} extends CI/CD into ML pipelines, while built-in drift detection and monitoring enable CT and CM. This creates strong feedback loops, where signals from production automatically inform retraining pipelines.

In terms of governance, GCP offers responsible AI toolkits for

fairness, bias detection, and explainability, though compliance certifications and governance maturity remain less extensive than Azure's. Logging and monitoring are integrated with Cloud Logging and Cloud Monitoring, but formal governance frameworks are still evolving.

For scalability, GCP leverages its strength in infrastructure, particularly tensor processing units (TPUs) for deep learning workloads. Elastic scaling across GPUs and CPUs is also available, making the platform well suited for large-scale experimentation.

Cost is relatively transparent; AutoML and managed training have predictable pricing, though costs can escalate quickly when automation is heavily used. Pretrained models for vision, speech, and natural language processing can offset costs by reducing the need for custom training. On openness, GCP supports major frameworks, with TensorFlow as its flagship.

Databricks

Databricks embodies an *Ops separation* mentality, explicitly distinguishing DataOps, ModelOps, and DevOps. Its Lakehouse architecture combines the scalability of data lakes with the reliability of data warehouses, structured through the medallion schema (bronze, silver, and gold layers) to manage raw, curated, and refined data. This design makes Databricks uniquely positioned to unify data engineering and ML pipelines.⁸

For version control, Databricks relies on open standards. Code can be versioned in Databricks Git Folders, while Delta Lake manages dataset versioning, lineage, and reproducibility. MLflow, a core component of Databricks, provides experiment tracking, artifact

^h Cloud Build: <https://cloud.google.com/build?hl=en>.

ⁱ Cloud Run Functions: <https://cloud.google.com/functions?hl=en>.

logging, and model registries, enabling teams to manage models with full lifecycle transparency.

Automation in Databricks spans the full MLOps lifecycle, combining native scheduling with flexible CI/CD integration. Lakehouse Jobs and Delta Live Tables automate ETL pipelines and model workflows. Databricks also supports CI/CD pipelines through APIs that integrate with external tools such as GitHub Actions, Azure DevOps, or Jenkins, enabling reproducible deployments of data and model artifacts. Beyond workflow orchestration, Databricks provides Databricks Asset Bundles and a Terraform provider to express infrastructure as code, automating the provisioning of jobs, serving endpoints, and feature pipelines. Together with MLflow for experiment tracking and model registry and the Feature Store for ensuring consistency between training and inference, Databricks creates reproducible feedback loops across data, model, and deployment pipelines.

Governance is supported by Delta Lake, which enforces data quality constraints and ensures lineage. MLflow adds model versioning and auditability, though enterprise compliance features are less extensive than Azure's or AWS's.

In terms of scalability, Databricks excels at handling large-scale data and ML workloads, leveraging its Spark-based foundation for distributed training and inference. Multicloud deployment is natively supported, allowing organizations to run Databricks pipelines across AWS, Azure, or GCP.

Cost is usage based, making it efficient for large data-intensive teams that can take advantage of its scaling capabilities. For smaller teams or less data-heavy workflows, Databricks

may be less cost-effective compared to turnkey managed services.

Databricks stands out for its strong platform flexibility. Its foundation on Spark, MLflow, and Delta Lake enables broad integration and reduces lock-in, while Git support, infrastructure as code, and multicloud deployment options improve portability across environments.

The Five Dimensions

Choosing an MLOps platform ultimately depends on aligning its philosophy with organizational priorities. Practitioners rarely face a purely technical decision as factors such as existing infrastructure (for example, Azure DevOps in enterprises or AWS ecosystems in startups) can be a decisive bias in the final choice. Yet other dimensions, such as performance, cost, openness, data management, and learning curve, can play equally critical roles. Each cloud provider makes different tradeoffs across these dimensions, so understanding them helps teams identify which platform best fits their context. In this work, we argue that these dimensions rotate around five factors.

Performance

Scalability and runtime efficiency differ across platforms. GCP's TPUs make Vertex AI attractive for deep learning workloads, while AWS offers Trainium and Inferentia accelerators to lower training and inference costs. Databricks, by contrast, is optimized for massive data pipelines rather than raw training speed, making it a strong candidate where data engineering is the bottleneck.

Cost

Cost models reflect the philosophies of each provider. AWS is highly flexible with granular pay-as-you-go

pricing, but smaller-scale projects often find it disproportionately expensive due to the overhead of combining multiple services. Azure is designed for long-term enterprise commitments, with cost efficiency improving as organizations integrate more deeply into the Microsoft ecosystem. GCP, in turn, tailors its pricing to AI and ML workloads; for example, AutoML and managed training jobs are straightforward to forecast, though they can escalate quickly when automation is heavily used. Databricks follows a usage-based model that favors data-heavy organizations, though it may be less cost-effective for smaller teams.

Openness

Portability and extensibility remain uneven. Databricks is the most open, built on Spark, MLflow, and Delta Lake, with strong multicloud support. In contrast, the platforms of other major vendors exhibit a simultaneous openness and an inherent vendor lock-in, where strengths in integration and automation can act as barriers to exiting the platform. AWS and Azure support open source frameworks but embed them in proprietary workflows, raising lock-in risks. GCP supports multiple frameworks but leans heavily on Google-native APIs and TensorFlow.

Data Management and DataOps

Databricks leads with its Lakehouse architecture and medallion schema, making it particularly strong for unifying data engineering and ML. AWS offers robust data management through S3 but requires stitching multiple services together. While GCP emphasizes automation over data-centric orchestration, Azure integrates data services through its

enterprise cloud and lately has been increasing its focus on data with the release of Fabric.⁹

Learning Curve

Azure is attractive for enterprises already using Microsoft tools, but new users may face a steep entry barrier. AWS offers depth and maturity, but its service sprawl increases operational complexity. GCP's automation reduces the need for expertise in early stages, but customization can become challenging. Databricks requires more configuration than ready-to-use services, but its reliance on open standards makes skills transferable across contexts.

The comparative analysis shows that no platform dominates across all dimensions. AWS provides unmatched breadth but is costly and complex at smaller scales. Azure excels in governance and long-term enterprise integration but limits cross-cloud portability. GCP lowers the barrier to adoption through automation and AutoML yet ties practitioners closely to Google's ecosystem. Databricks stands out for openness and data-centric integration but requires more configuration effort. For practitioners, the decision is less about features than about tradeoffs across performance, cost, openness, data management, and the learning curve.

MLOps in the cloud is not yet settled. Practitioners should view platform choice as more than tooling; it is a commitment to a philosophy of how data, models, and operations will evolve together.

Actionable Insights

- *Choose platforms by their philosophy:* For example, choose Azure

ABOUT THE AUTHORS



SERGIO MORESCHINI is a postdoctoral researcher at Tampere University, 33720 Tampere, Finland, and the University of Oulu, 90550 Oulu, Finland. Contact him at sergio.moreschini@tuni.fi.



DAVID HÄSTBACKA is an associate professor in software engineering at Tampere University, 33720 Tampere, Finland. Contact him at david.hastbacka@tuni.fi.



DAVIDE TAIBI is a professor of software engineering at the Southern Denmark University, 7100 Vejle, Denmark, and at the University of Oulu, 90550 Oulu, Finland. Contact him at davide@taibi.it.

for governance, AWS for ecosystem breadth, GCP for automation, and Databricks for openness and DataOps.

- *Balance costs with scale:* AWS is flexible but costly at a small scale, Azure fits long-term enterprise use, GCP targets AI/ML workloads, and Databricks favors data-heavy teams.
- *Mind lock-in risks:* AWS, Azure, and GCP tie open frameworks to proprietary services, while Databricks offers the highest openness and multicloud portability. 

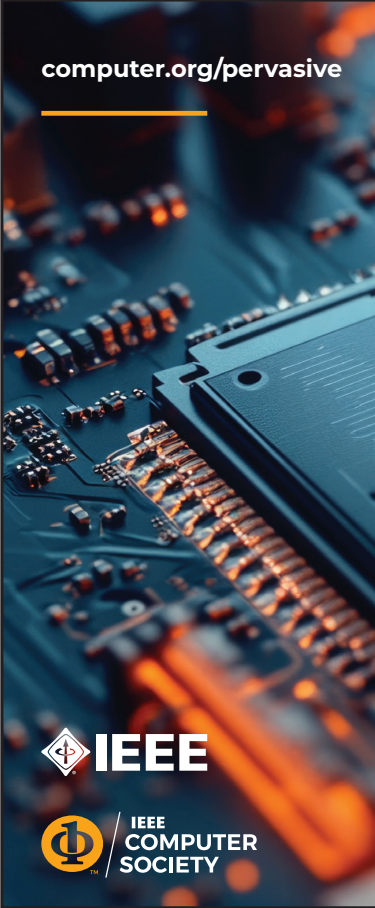
Acknowledgment

This work has been funded by Business Finland (Grant 6GSoft and IndustryX).

References

1. S. Moreschini, F. Lomio, D. Hästbacka, and D. Taibi, "MLOps for evolvable AI intensive software systems," in *Proc. IEEE Int. Conf. Softw. Anal., Evol. Reengineering (SANER)*, Piscataway, NJ, USA: IEEE Press, 2022, pp. 1293–1294, doi: [10.1109/SANER53432.2022.00155](https://doi.org/10.1109/SANER53432.2022.00155).
2. Cloud Architecture Center. "MLOps: Continuous delivery and automation pipelines in machine learning." Google. Accessed: Nov. 25, 2025. [Online]. Available: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
3. "What is azure policy?" Microsoft Learn. Accessed: Nov. 25, 2025.

- [Online]. Available: <https://learn.microsoft.com/en-us/azure/governance/policy/overview>
4. P. D. Francesco and E. Sela, "Track your ml experiments end to end with data version control and amazon SageMaker experiments," *aws*, Jul. 14, 2022. [Online]. Available: <https://aws.amazon.com/blogs/machine-learning/track-your-ml-experiments-end-to-end-with-data-version-control-and-amazon-sagemaker-experiments/>
 5. M. Hardt et al., "Amazon SageMaker clarify: Machine learning bias detection and explainability in the cloud," in *Proc. 27th ACM SIG-KDD Conf. Knowl. Discovery Data Mining (KDD)*, New York, NY, USA: ACM, 2021, pp. 2974–2983, doi: [10.1145/3447548.3467177](https://doi.org/10.1145/3447548.3467177).
 6. S. Chookshi, S. Mckinnon, D. Turchiarelli, L. Kozhushnik, and D. Crawford, "Machine learning operations." Microsoft Learn. Accessed: Nov. 25, 2025. [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/machine-learning-operations-v2?utm_source=chatgpt.com
 7. "MLOps on vertex AI." Google. Accessed: Nov. 25, 2025. [Online]. Available: <https://cloud.google.com/vertex-ai/docs/start/introduction-mlops>
 8. "MLOps workflows on databricks." Databricks. Accessed: Nov. 25, 2025. [Online]. Available: <https://docs.databricks.com/aws/en/machine-learning/mlops/mlops-workflow>
 9. C. Xu, "Dataops in fabric data factory," *Fabric Blog*, Apr. 3, 2025. [Online]. Available: <https://blog.fabric.microsoft.com/en-US/blog/data-operations-in-fabric-data-factory/>



[computer.org/pervasive](https://www.computer.org/pervasive)



IEEE



**IEEE
COMPUTER
SOCIETY**



pervasive COMPUTING

Call for Articles

IEEE Pervasive Computing publishes accessible, useful peer-reviewed papers on the latest developments in pervasive, mobile, and ubiquitous computing. Topics include hardware technology, software infrastructure, real-world sensing and interaction, human-computer interaction, and systems considerations, including deployment, scalability, security, and privacy.



Author Guidelines

bit.ly/3UoGDT7